#itemID : string #name : string #data : map<string, string> #WorkItem(itemID : string, name : string) #WorkItem(other : WorkItem &const) +~WorkItem() +getItemID() : string +getName() : string +setData(key : string, value : string) : void +getData(key : string) : string +getChildren() : vector<WorkItem*> +hasData(key : string) : bool +makeAvailable() : void +assign(participant : Participant*) : void +start() : void +complete() : void +reject(reason : string &const) : void +cancel() : void +isFinished() : bool const +isAvailable() : bool const +clone() : WorkItem *const +createIterator() : WorkItemIterator* +createMemento() : TaskMemento* +restoreMemento(memento : TaskMemento*) : void +getStateName() : string const +createAvailableIterator() : WorkItemIterator* +hasData(key : string &const) : bool +getAsignee() : Participant *const +activateChildren() : void +setPublisher(publisher : EventPublisher*) : void WorkItem {Composite/Decorator = "Component"} {Iterator = "Aggregate"} {Command = "Reciever"} {Memento = "Originator"} -currentState : TaskState* -asignee : Participant* -requiredRole : Role* -strategy : AssignmentStrategy* -deadline : time_t -publisher : EventPublisher* +Task(other : Task&) +~Task() +makeAvailable() : void +assign(participant : Participant*) +start() : void +complete() : void +reject(reason : string &const) : void +cancel() : void +isFinished() : bool const +isAvailable() : bool const +getStateName() : string const +handleChange(newState : TaskState*) +clone() : WorkItem *const +createMemento() : TaskMemento* +restoreMemento(memento : TaskMemento*) : void +getAssignee() : Participant* +setAssignee(participant : Participant*) : void +getRequiredRole() : Role* +setAssignmentStrategy(strategy : AssignmentStrategy*) : void +autoAssign(candidates : vector<Participant*>) +Task(intemID : string &const, name : string &const, requiredRole : Role*, strategy : AssignmentStrategy*) +setDeadline(deadline : time_t) : voif +getDeadline() : time_t +setPublisher(publisher : EventPublisher*) : voif +getPublisher() : EventPublisher *const Task {Composite = "Leaf"} {State = "Context"} {Strategy = "Context"} {Decorator = "Concrete Component"} {Facade = "Subsystem"} #children : vector<WorkItem*> #activated_ : bool #Stage(itemID : string &const, name : string &const) #Stage(other : Stage &const) +add(item : WorkItem*) : void +getChildren() : vector<WorkItem*> +makeAvailable() : void +assign(participant : Participant*) : void +start() : void +complete() : void +reject(reason : string &const) : void +cancel() : void +isFinished() : bool const +isAvailable() : bool const +getStateName() : string const +createMemento() : TaskMemento* +restoreMemento(memento : TaskMemento*) : void +activateChildren() : void +~Stage() Stage +WorkFlowBuilder() +~WorkFlowBuilder() +getResult() : WorkflowDefinition* +reset(name : string, version : int) : void +beginSequentialStage(name : string) : void +beginParallelStage(name : string) : void +addTask(name : string, role : Role*, strategy : AssignmentStrategy*) : void +addExternalTask(name : string, connector : ExternalConnector*, request : string, role : Role*, strategy : AssignmentStrategy*) : void +withAudit(publisher : EventPublisher*) : void +withValidation(fields : vector<string>) +withPriority(level : int) : void +endStage() : void WorkFlowBuilder -definition : WorkflowDefinition* -stageStack : stage<Stack*> -pendingTask : WorkItem* -nextItemID : int +DefinitionBuilder() +~DefinitionBuilder() +getResult() : WorkflowDefinition* +reset(name : string, version : int) : void +beginSequentialStage(name : string) : void +beginParallelStage(name : string) : void +addTask(name : string, role : Role*, strategy : AssignmentStrategy*) : void +addExternalTask(name : string, connector : ExternalConnector*, request : string, role : Role*, strategy : AssignmentStrategy*) : void +withAudit(publisher : EventPublisher*) : void +withValidation(fields : vector<string>) +withPriority(level : int) : void +endStage() : void +commitPendingTask() : void DefinitionBuilder -builder : WorkflowBuilder& +Director(builder : WorkflowBuilder&) +construct(name : string &const, version int, recipe : function<void(WorkflowBuilder&)>&) : void +~Director() Director +~WorkItemIterator() +first() : void +next() : void +isDone() : bool +current() : WorkItem* WorkItemIterator -root : WorkItem* -pending : stack<WorkItem*> -currentItem : WorkItem* +DepthFirstIterator(root : WorkItem*) +first() : void +next() : void +isDone() : bool +current() : WorkItem* +~DepthFirstIterator() DepthFirstIterator -inner : DepthFirstIterator +AvailableItemsIterator(root : WorkItem*) +first() : void +next() : void +isDone() : bool +current() : WorkItem* +skipToAvailable() : void +~AvailableItemsIterator() AvailableItemsIterator -currentChild_ : WorkItem* -currentActivated_ : bool +SequentialStage(itemID : string, name : string) +SequentialStage(other : const SequentialStage&) +activateChildren() : void +clone() : WorkItem *const +~SequentialStage() SequentialStage +~TaskState() +assign(task : Task*, participant : Participant*) : void +start(task : Task*) : void +complete(task : Task*) : void +reject(task : Task*, reason : string) : void +cancel(task : Task*) : void +makeAvailable(task : Task*) : void +getName() : string const +clone() : TaskState *const +isFinal() : bool const +isAvailable() : bool const TaskState +makeAvailable(task : Task*) : void +getName() : string const +clone() : TaskState *const +cancel(task : Task*) : void CreatedState +assign(task : Task*, participant : Participant*) : void +reject(task : Task*, reason : string) : void +cancel(task : Task*) : void +getName() : string const +clone() : TaskState *const +isAvailable() : bool const AvailableState+complete(task : Task*) : void +reject(task : Task*, reason : string) : void +cancel(task : Task*) : void +getName() : string const +clone() : TaskState *const StartedState +getName() : string const +clone() : TaskState *const +isFinal() : bool CompletedState +getName() : string const +clone() : TaskState *const +isFinal() : bool const CancelledState +cancel(task : Task*) : void +makeAvailable(task : Task*) : void +getName() : string const +clone() : TaskState *const RejectedState +selectParticipant(task : Task*, participants : vector<Participant*>) : Participant* +~AssignmentStrategy() AssignmentStrategy +selectParticipant(task : Task*, participants : vector<participant*>&) : Participant* +hasRequiredRole(participant : Participant*, task : Task*) : bool RoleBasedAssignment +selectParticipant(task : Task*, participants : vector<Participant*>) : Participant* WorkloadAssignment -rule : function<bool(Task*,Participant*)> +RuleBasedAssignment(rule : function<bool(Task*,Participant*)>) +selectParticipant(task : Task*, participants : vector<Participant*>) : Participant* +evaluateRule(task : Task*, participant : Participant*) : bool +setRule(rule : function<bool(Task*,Participant*)>) +~RuleBasedAssignment() RuleBasedAssignment -next : ApprovalHandler* +~ApprovalHandler() +setNext(handler : ApprovalHandler*) +handle(request : ApprovalRequest*) #forward(request : ApprovalRequest*) ApprovalHandler {Facade = "Subsystem"} -role : Role* -timeoutSeconds : int +ApprovalLevel(role : Role*, timeoustSeconds : int) +~ApprovalLevel() +canApprove(request : const ApprovalRequest*) : bool +handle(request : ApprovalRequest*) : void +getRole() : Role* +getTimeout() : int ApprovalLevel -subject : WorkItem* -requiredLevel : int -status : ApprovalStatus -createdAt : time_t -deadline : time_t -decidedBy : Role* +ApprovalRequest(subject : WorkItem*, requiredLevel : int, deadline : time_t) +~ApprovalRequest() +getSubject() : WorkItem *const +getRequiredLevel() : int const +getStatus() : ApprovalStatus const +getDeadline() : time_t const +resetDeadline(deadline : time_t) : void +approve(by : Role*) : void +reject(by : Role*) : void +markEscalated() : void +isExpired(now : time_t) : bool const ApprovalRequest -ListenerList : vector<WorkflowListener*> +EventPublisher() +attach(listener : WorkflowListener*) : void +detach(listener : WorkflowListener*) : void +publish(event : WorkflowEvent*) : void +~EventPublisher() EventPublisher {Facade = "Subsystem"} +WorkflowListener() +update(event : WorkflowEvent*) : void +~WorkflowListener() WorkflowListener -instance : WorkflowInstance* -approvalChain : ApprovalHandler* +EngineRouter(instance : WorkflowInstance*, approvalChain : ApprovalHandler*) +update(event : WorkflowEvent*) : void +~EngineRouter() EngineRouter -sentMessages : vector<string> +AuditLogger() +update(event : WorkflowEvent*) : void +getRecords() : vector<string> +~AuditLogger() AuditLogger -notifications : vector<string> +Notifier() +update(event : WorkflowEvent*) : void +getNotifications() : vector<string> +getNotificationCount() : int const +clearNotifications() : void +~Notifer() Notifier -transitionCounts : map<string, int> -stateEntryTime : map<string, string> -totalTimeInState : map<string, float> -currentState : map<string, string> -completedIntervals : map<string, int> +MetricsCollector() +update(event : WorkflowEvent*) : void +getTransitionCount(state : string) : int +getAverageTimeInState(state : string) : float +getSummary() : string +~MetricsCollector() MetricsCollector #item : WorkItem* #backup : TaskMemento* #WorkFlowCommand(item : WorkItem*) +execute() : void +undo() : void +~WorkFlowCommand() +getDescription() : string const #saveBackup() : void #restoreBackup() : void WorkFlowCommand -participant : Participant* +AssignCommand(item : WorkItem*, participant : s... +execute() : void +undo() : void +getDescription() : string const +~AssignCommand() AssignCommand +CompleteCommand(item : WorkItem*) +execute() : void +undo() : void +~CompleteCommand() +getDescription() : string const CompleteCommand -reason : string +RejectCommand(item : WorkItem*, reason : string) +execute() : void +undo() : void +getDescription() : string const +~RejectCommand() RejectCommand -state : TaskState* -asignee : Participant* -data : map<string, string> +~TaskMemento() -TaskMemento(state : TaskState*, assignee : Participant*, data : map<string, string>) TaskMemento -commands : vector<WorkflowCommand*> +CommandHistory() +push(command : WorkflowCommand*) : void +pop() : WorkflowCommand* +isEmpty() : bool +size() : int +~CommandHistory() CommandHistory {Facade = "Subsystem"} -task : WorkItem* +TaskDecorator(task : WorkItem*) +getItemID() : string +getName() : string +setData(key : string, value : string) : void +getData(key : string) : string +hasData(key : string) : bool +makeAvailable() : void +isAvailable() : bool const +TaskDecorator() +assign(participant : Participant*) : void +start() : void +complete() : void +reject(reason : string &const) : void +cancel() : void +isFinished() : bool const +clone() : WorkItem *const +createIterator() : WorkItemIterator* +createMemento() : TaskMemento* +restoreMemento(memento : TaskMemento*) : void +getStateName() : string const +createAvailableIterator() : WorkItemIterator* +getChildren() : vector<WorkItem*> TaskDecorator -requiredFields : vector<string> +ValidatedTask(task : WorkItem*, fields : vector<string>) +ValidatedTask(other : ValidatedTask &const) +~ValidatedTask() +validate() : bool +start() : void +complete() : void +addRequiredField(field : string &const) : void +clone() : WorkItem *const ValidatedTask -publisher : EventPublisher* +AuditedTask(task : WorkItem*, publisher : EventPublisher*) +AuditedTask(other : AuditedTask &const) +makeAvailable() : void +assign(participant : Participant*) : void +start() : void +complete() : void +reject(reason : string &const) : void +cancel() : void +clone() : WorkItem *const +setPublisher(publisher : EventPublisher*) : void +~AuditedTask() -audit(action : string) : void AuditedTask -priority : int +PriorityTask(task : WorkItem*, priority : int) +PriorityTask(other : PriorityTask &const) +setPriority() : void +getPriority() : int const +clone() : WorkItem *const +~PriorityTask() PriorityTask -definitions : map<string, WorkflowDefinition*> -instances : map<string, WorkflowInstance*> -participants : map<string, Participant*> -routers : vector<EngineRouter*> -invoker : CommandInvoker -audit : AuditLogger -notifier : Notifier -metrics : MetricsCollector -nextInstanceNumber : int +~TIVIDYFacade() +TIVIDYFacade() +createInstance(definitionName : string) : string +registerDefinition(definition : WorkflowDefinition*) : void +registerParticipant(participant : Participant*) : void +assignTask(instanceID : string &const, taskID : string &const, participant : string &const) : void +completeTask(instanceID : string &const, taskID : string &const) : void +rejectTask(instanceID : string &const, taskID : string &const, reason : string &const) : void +undoLastAction() : bool +requestApproval(instanceID : string &const, taskID : string &const) : ApprovalStatus +getMetricsSummary() : string +getAuditLog() : vector<string> +findItem(instanceID : string &const, itemID : string &const) : WorkItem* +getParticipant() : Participant *const +isTaskAssignedTo(instanceID : string &const, taskID : string &const, participantID : string &const) : bool +startInstance(instanceID : string &const) : void +startTask(instanceID : string &const, taskID : string &const) -findInstance(instanceID : string &const) : WorkflowInstance* -findParticipant(participantID : string &const) : Participant* -findItem(itemID : string &const) : WorkItem* TIVIDYFacade +ParalellStage(itemID : string, name : string) +ParalellStage(other : const ParallelStage&) +activateChildren() : void +clone() : WorkItem *const +~ParalellStage() ParalellStage -root : WorkItem* -name : string -version : int -approvalChain : ApprovalHandler* -frozen : bool +WorkflowDefinition(name : string &const, version : int) +getRoot() : WorkItem *const +start() : void +~WorkflowDefinition() +getName() : string const +getVersion() : int const +setRoot(root : WorkItem*) : void +setApprovalChain() : ApprovalHandler* +freeze() : void +isFrozen() : bool +createInstance(instanceID : string) : WorkflowInstance* +getApprovalChain() : ApprovalHandler *const WorkflowDefinition {Facade = "Subsystem"} -instanceID : string -definition : WorkflowDefinition* -rootWorkItem : WorkItem* -status : WorkflowStatus -outcome : string -publisher : EventPublisher* +WorkflowInstance(instanceID : string &const, definition : WorkFlowDefinition*) +getInstanceID() : string const +getDefinitionVersion() : int const +getRoot() : WorkItem *const +getStatus() : WorkflowStatus const +getOutcome() : string const +setOutcome(outcome : string) : void +start() : void +cancel() : void +isFinished() : bool +activatedNext() : void +getAvailableItems() : vector<WorkItem*> +getPublisher() : EventPublisher *const +getDefinition() : WorkflowDefinition *const +fail(reason : string &const) : void +finish(finalStatus : WorkflowStatus, defaultOutcome : string &const) +connectPublisher() : void WorkflowInstance +ExternalConnector() +request(operation : string *const, payload : string *const) : void +getName() : string +~ExternalConnector()ExternalConnector -entries : Map<string, string> -ServiceName : string +APIOne() +addEntry(key : string &const, value : string &const) : void +removeEntry(key : string &const) : void +~APIOne() APIOne -description : string -address : string +APITwo() +add() : void +alert() : void +setInfo(param : string &const, Address : string &const) : void +dispatch() : void +~APITwo() APITwo -adaptee : APIOne* +AdapterOne(adaptee : APIOne*) +request(operation : string *const, payload : string *const) : void +getName() : string +~AdapterOne() AdapterOne +start(task : Task*) : void +reject(task : Task*, reason : string) : void +cancel(task : Task*) : void +getName() : string const +clone() : TaskState *const AssignedState -partipantID : string -name : string -role : Role* -workload : int +Participant(participantID : string, name : string, role : Role*) +getParticipantID() : string const +getName() : string const +getRole() : Role *const +getWorkload() : int const +incrementWorkload() : void +decrementWorkload() : void +hasRole(role : Role*) : bool Participant -name : string -authorityLevel : int -permissions : vector<string> +Role(name : string, authorityLevel : int) +getName() : string const +getAuthorityLevel() : int const +equals(other : Role*) : bool const +addPermission(permission : string &const) : void +hasPermission(permission : string &const) : bool const +~Role() Role -history : CommandHistory +CommandInvoker() +executeCommand(command : WorkflowCommand*) : void +undoLast() : bool +canUndo() : bool const +getHistory() : CommandHistory* +~CommandInvoker()CommandInvoker -connector : ExternalConnector* -request : string -response : string +ExternalTask(itemID : string, name : string, role : Role*, strategy : AssignmentStrategy*, connector : ExternalConnector*, request : string) +clone() : WorkItem *const +setRequest(request : string const) : void +execute() : bool +getResponse() : string +~ExternalTask() ExternalTask -type : EventType -sourceID : string -timestamp : string -details : string -approvalRequest : ApprovalRequest* +WorkflowEvent(type : EventType, sourceID string, timestamp : string, details : string) +getType() : EventType +getSourceID() : string +getTimestamp() : string +getDetails() : string +~WorkflowEvent() +getApprovalRequest() : ApprovalRequest* WorkflowEvent Composite Builder Iterator State Strategy Chain of Responsibility Observer Command Memento Decorator Adapter Prototype Facade +startInstance(instanceID : string, userID : string) : void +assignTask(instanceID : string, taskID : string, participantID : string, userID : string) : void +startTask(instanceID : string, taskID : string, userID : string) : void +completeTask(instanceID : string, taskID : string, userID : string) : void +rejectTask(instanceID : string, taskID : string, reason : string, userID : string) : void +requestApproval(instanceID : string, taskID : string, userID : string) : ApprovalStatus +~IWorkflowService() IWorkflowService {Proxy = "Subject"} Proxy -adaptee : APITwo* +AdapterTwo(adaptee : APITwo*) +request(operation : string *const, payload : string *const) : void +getName() : string +~AdapterTwo() AdapterTwo2 +Pending +Approved +Rejected +Escalated ApprovalStatus << enumeration >> -realService : TIVIDYFacade* -hasPermission(userID : string, permission : string) : bool -ownsTask(instanceID : string, taskID : string, userID : string) : bool -requirePermission(userID : string, permission : string) : void -requireOwnership(instanceID : string, taskID : string; userID:string) : void +~SecureWorkflowProxy() +SecureWorkflowProxy() +startInstance(instanceID : string, userID : string) : void +assignTask(instanceID : string, taskID : string, participantID : string, userID : string) : void +startTask(instanceID : string, taskID : string, userID : string) : void +completeTask(instanceID : string, taskID : string, userID : string) : void +rejectTask(instanceID : string, taskID : string, reason : string, userID : string) : void +requestApproval(instanceID : string, taskID : string, userID : string) : ApprovalStatus SecureWorkflowProxy {Proxy = "Proxy"} +TEM_STATE_CHANGED +ITEM_COMPLETED +TASK_ACTION +EXTERNAL_EVENT +TIMEOUT +WORKFLOW_FINISHED EventType <<enumeration>> +StartCommand(item : WorkItem*) +execute() : void +undo() : void +getDescription() : string const +~StartCommand()StartCommand -CREATED -RUNNING -COMPLETED -CANCELLED -FAILED WorkflowStatus <<enumeration >> 1 1 1 0..* 1 1 1 0..* machine 0..* * 0..1 0..* Adaptee 1:1 Adaptee component <<friend>> Observer list successor State <<instantiate>> Children Chain Visual Paradigm Standard(Taya Govender(University of Pretoria))